

IT3130 Application Development

Group viva, live Swagger demo, and per-member commands

RideLink

Viva & Demo Playbook (four members)

Print this PDF. Each member keeps their own section open. There is no website. Swagger UI is the official client in the brief. All commands below were verified against this repository.

Module	IT3130 Application Development
Assessment	Group assignment 30% + scheduled demonstration / viva
Due	01/10/2026
Repo	https://github.com/IT24104300/RideLink-AD-
Stack	Java 17 Spring Boot 3.4 Maven Wrapper H2 per service
Demo UI	Chrome http://127.0.0.1:PORT/swagger-ui.html
Users	passenger1 / driver1 / admin1 password = password

Who does what on the viva

Member 1	Account Service port 8081 login, JWT, /me
Member 2	Driver & Vehicle port 8082 profile, eligible list
Member 3	Ride Management port 8083 create, assign, lifecycle (the story)
Member 4	Fare & Payment port 8084 estimate, pay 4242, receipt

Not leaked exam papers. These are the clicks, commands, and JSON bodies a marker expects if they grade architecture, REST, JWT, tests, Git, and a live Swagger demo.

1. What to do on the viva day

It is a group presentation AND an individual viva. Show the running APIs in Chrome Swagger, then each person is questioned. A taxi website is not required. If they ask 'where is the UI?', answer: Swagger is the UI the brief asked for.

Suggested 12-15 minute flow

- 0:00-1:00 All four Java processes already running. One laptop, Chrome, four tabs.
- 1:00-2:30 Any member (often Member 3) explains: four services, four DBs, JWT.
- 2:30-5:00 Member 1 Account: login, Authorize, GET /me.
- 5:00-7:00 Member 2 Driver: GET /me, eligible?pickup=Colombo Fort.
- 7:00-10:00 Member 3 Ride: create, assign, accept, start, complete.
- 10:00-12:00 Member 4 Fare estimate, pay 4242, receipt.
- 12:00-13:30 One negative: 401, 409 Jaffna, 400 REQUESTED-complete, or card 0000.
- 13:30+ Individual questions. Everyone must explain the WHOLE system.

Rules while presenting

- Use Chrome or Edge. Do not use Cursor Simple Browser.
- Pickup must include the word Colombo so seeded driver1 is eligible.
- Switching passenger/driver means LOGIN AGAIN and Authorize with the NEW token.
- Swagger Authorize: paste the token only. Do not type the word Bearer.
- Authorize is per browser tab. Repeat it on 8082, 8083 and 8084.

2. Start everything BEFORE they sit down

From the repository root in PowerShell. Four terminals. Order: Account, Driver, Fare, then Ride.

```
$env:JAVA_HOME = "C:\Program Files\Microsoft\jdk-21.0.11.10-hotspot"
cd C:\Users\MSI\Desktop\Pro

.\mvnw.cmd -pl services/account-service -am spring-boot:run
.\mvnw.cmd -pl services/driver-vehicle-service -am spring-boot:run
.\mvnw.cmd -pl services/fare-payment-service -am spring-boot:run
.\mvnw.cmd -pl services/ride-service -am spring-boot:run
```

Wait until each terminal prints Started ...Application. Then check health (expect {"status":"UP"}):

```
http://127.0.0.1:8081/actuator/health
http://127.0.0.1:8082/actuator/health
http://127.0.0.1:8083/actuator/health
http://127.0.0.1:8084/actuator/health
```

IntelliJ IDEA (same for every member)

- File -> Open -> the root pom.xml (whole RideLink project, not one service folder).
- Project SDK = JDK 17 or 21. Do NOT run the parent module.
- Run only YOUR main class:

Member 1	com.ridelink.account.AccountServiceApplication
Member 2	com.ridelink.driver.DriverVehicleServiceApplication
Member 3	com.ridelink.ride.RideServiceApplication
Member 4	com.ridelink.fare.FarePaymentServiceApplication

Tests before the viva (whole project):

```
.\mvnw.cmd -q test
```

Opening lines (any member, 60 seconds)

SAY: "RideLink is a fictional ride-sharing backend. We split it into four Spring Boot microservices in Java: Account, Driver and Vehicle, Ride, and Fare and Payment. Each service owns its own H2 database. Account issues a JWT; the others validate the same secret and enforce passenger, driver, and admin roles. We demo with Swagger, because the brief is backend-only."

3. Member 1 — Account Service (8081)

Folder	services/account-service
Main class	com.ridelink.account.AccountServiceApplication
Swagger	http://127.0.0.1:8081/swagger-ui.html
Open in IDE	AccountController.java AccountService.java AccountServiceTest.java

Commands

```
.\mvnw.cmd -pl services/account-service -am spring-boot:run
.\mvnw.cmd -pl services/account-service -am test
```

What you own

- POST /api/accounts/register — passenger or driver only (ADMIN self-register is rejected).
- POST /api/accounts/login — returns accessToken (JWT).
- GET / PUT /api/accounts/me — profile.
- Admin: GET /api/accounts/{id} PATCH /api/accounts/{id}/status

What you say (45 seconds)

SAY: "I own Account Service. Passwords are stored as BCrypt hashes, never plaintext. Login signs a JWT with the account UUID, username, and role. Other services never open my database; they only read the token."

Code / JSON you apply in Swagger

1. POST /api/accounts/login

```
{ "username": "passenger1", "password": "password" }
```

Copy accessToken. Click Authorize (green lock). Paste the token. Do not type Bearer.

2. GET /api/accounts/me — must return passenger1.

3. Optional register (role PASSENGER or DRIVER only):

```
{
  "username": "newpass1",
  "password": "password1",
  "email": "newpass1@ridelink.local",
  "fullName": "New Passenger",
  "phone": "0771111111",
  "role": "PASSENGER"
}
```

4. Negative: clear Authorize, GET /api/accounts/me -> 401 UNAUTHORIZED.

5. Optional admin: login as admin1, then PATCH status:

```
{ "status": "SUSPENDED" }
```

Likely questions

Q: Why JWT instead of sessions?

A: Four separate apps. A session cookie on one server is not trusted by the others. A signed token is validated locally with the shared secret.

Q: What is in the token?

A: sub = account UUID, username, role (PASSENGER / DRIVER / ADMIN), expiry. Signed HS256.

Q: Can a passenger register as ADMIN?

A: No. Self-registration of ADMIN is rejected. Admin accounts are seeded.

4. Member 2 — Driver & Vehicle Service (8082)

Folder	services/driver-vehicle-service
Main class	com.ridelink.driver.DriverVehicleServiceApplication
Swagger	http://127.0.0.1:8082/swagger-ui.html
Open in IDE	DriverController.java DriverProfileService.java DriverEligibilityFilter.java

Commands

```
.\mvnw.cmd -pl services/driver-vehicle-service -am spring-boot:run
.\mvnw.cmd -pl services/driver-vehicle-service -am test
```

What you own

- PUT / GET /api/drivers/me — vehicle, availability, service area, simulated location.
- GET /api/drivers/eligible?pickup= — called by Ride Service on assign.
- Seeded driver1: plate CAB-1234, area Colombo, available true.

What you say

SAY: "I own operational driver data, not login. Eligible means available, plate present, and pickup matching the service area. Distance is a dummy string hash, not GPS. Ride Service calls my eligible API. It must not query my tables."

Code / JSON you apply in Swagger

First: Member 1 logs in as driver1 on 8081. You Authorize 8082 with that token.

1. GET /api/drivers/me — expect CAB-1234 and Colombo.
2. Optional PUT /api/drivers/me

```
{
  "displayName": "Demo Driver",
  "vehicleMake": "Toyota",
  "vehicleModel": "Prius",
  "vehiclePlate": "CAB-1234",
  "vehicleColor": "White",
  "available": true,
  "serviceArea": "Colombo",
  "locationLabel": "Colombo Fort",
  "latitude": 6.93,
  "longitude": 79.85
}
```

3. GET /api/drivers/eligible?pickup=Colombo Fort — list must include driver1.
4. Negative: set available false, or Ride assign with pickup Jaffna -> 409.

Likely questions

Q: Why not inside Account Service?

A: Account is identity. This is operations: vehicle, shift, area. Separate data owner.

Q: How is a driver eligible?

A: available == true, vehicle plate present, pickup contains serviceArea or locationLabel (ignore case). Then sort by dummy distance.

Q: Is location real GPS?

A: No. Simulated labels. The brief forbids live maps.

5. Member 3 — Ride Service (8083)

Folder	services/ride-service
Main class	com.ridelink.ride.RideServiceApplication
Swagger	http://127.0.0.1:8083/swagger-ui.html
Open in IDE	RideController.java RideService.java RideStatus.java RideServiceTest.java

Commands

```
.\mvnw.cmd -pl services/ride-service -am spring-boot:run
.\mvnw.cmd -pl services/ride-service -am test
```

What you own

- POST /api/rides — passenger creates REQUESTED.
- POST /api/rides/{id}/assign — REST call to Driver eligible API.
- accept -> start -> complete (driver). complete calls Fare /api/fares/final.
- POST /api/rides/{id}/payment — Fare Service attaches paymentId after pay.

What you say

SAY: "I own the ride lifecycle. States are REQUESTED, ASSIGNED, ACCEPTED, IN_PROGRESS, COMPLETED, and CANCELLED. Invalid jumps return 400 INVALID_TRANSITION. On assign I call Driver Service. On complete I call Fare Service. I store UUIDs only — never their databases."

Code / JSON you apply in Swagger

Passenger token. POST /api/rides

```
{ "pickup": "Colombo Fort", "destination": "Kandy" }
```

Copy id. POST /api/rides/{id}/assign

```
{ }
```

Member 1 logs in as driver1. You Authorize 8083 with the driver token, then:

```
POST /api/rides/{id}/accept
POST /api/rides/{id}/start
POST /api/rides/{id}/complete
```

GET /api/rides/{id} — status COMPLETED, fareId set. After Member 4 pays, paymentId is set.

Negative A — no driver. New ride:

```
{ "pickup": "Jaffna", "destination": "Kandy" }
```

Then POST assign -> 409 NO_DRIVER_AVAILABLE.

Negative B — invalid transition. New Colombo ride (still REQUESTED), then POST complete -> 400 INVALID_TRANSITION.

Likely questions

Q: Draw the state machine.

A: REQUESTED -> ASSIGNED -> ACCEPTED -> IN_PROGRESS -> COMPLETED. Any of the first four may go to CANCELLED. COMPLETED and CANCELLED are terminal.

Q: Why REST to Driver, not a queue?

A: Assign must return the chosen driver in the same HTTP response. A queue would be eventually consistent and harder to demo.

Q: What if Fare is down on complete?

A: The ride still becomes COMPLETED and fareNote explains the failure.

6. Member 4 — Fare & Payment Service (8084)

Folder	services/fare-payment-service
Main class	com.ridelink.fare.FarePaymentServiceApplication
Swagger	http://127.0.0.1:8084/swagger-ui.html
Open in IDE	FarePaymentController.java FareCalculator.java FarePaymentService.java

Commands

```
.\mvnw.cmd -pl services/fare-payment-service -am spring-boot:run
.\mvnw.cmd -pl services/fare-payment-service -am test
```

What you own

- POST /api/fares/estimate and POST /api/fares/final
- POST /api/payments — simulated. cardLast4 0000 or simulateFailure true -> FAILED.
- GET /api/payments/{id}/receipt
- After a successful pay, this service calls Ride POST /api/rides/{id}/payment

What you say

SAY: "Fares use a documented formula: total = 150 + (km times 80) + (minutes times 5) in LKR. Distance is simulated from the pickup and destination strings — no Google Maps. Payments are fake records. A successful pay tells Ride Service the paymentId."

Code / JSON you apply in Swagger

Passenger token. POST /api/fares/estimate

```
{ "pickup": "Colombo Fort", "destination": "Kandy" }
```

Show total and the formula field. After Member 3 completes the ride, POST /api/payments

```
{
  "rideId": "<paste ride id>",
  "fareId": "<paste fare id from complete>",
  "amount": 813.30,
  "currency": "LKR",
  "method": "CARD",
  "cardLast4": "4242"
}
```

Use the real amount from estimate/complete (often 813.30 for Colombo Fort -> Kandy). Then GET /api/payments/{id}/receipt.

Fail demo — same body but:

```
{ "cardLast4": "0000" }
```

Status FAILED. Ride paymentId is not attached.

Likely questions

Q: Write the fare formula.

A: total = base + (distanceKm * perKm) + (durationMin * perMin). Defaults 150, 80, 5. Distance hashed from place names into about 2-20 km.

Q: Why not Stripe?

A: The brief requires simulated payments only.

Q: What if Ride is down when you pay?

A: The payment row still saves. Attach can be retried.

7. Integrated happy path (all four watching)

Four Chrome tabs. Pickup always Colombo Fort. Destination Kandy.

- M1: login passenger1 on 8081. Authorize 8081, 8083, 8084 with that token.
- M4: POST /api/fares/estimate.
- M3: POST /api/rides then POST assign.
- M1: login driver1. Give the new token to M2 and M3.
- M2: GET /api/drivers/eligible?pickup=Colombo Fort.
- M3: accept, start, complete.
- M1: login passenger1 again. Authorize 8084 and 8083.
- M4: pay 4242. M3: GET ride — paymentId filled.
- Anyone: 401 without token, or 409 pickup Jaffna, or complete a REQUESTED ride (400).

8. Group questions (every member)

Q: Why four services, not a monolith?

A: The brief requires four independently owned business services. A monolith is simpler (one deploy, one DB) but fails G1. Cost: extra JWT copies, no joins, partial failures. We accept that and use timeouts plus UUID references.

Q: Does an API Gateway count as a service?

A: No. Gateway, Eureka, or config server must not replace one of the four core business services.

Q: How do services talk?

A: Synchronous REST. Ride to Driver eligible; Ride to Fare final; Fare to Ride attach payment. REST fits Swagger/Postman and request-response workflows. Async is documented as a later option.

Q: How do you avoid a shared database?

A: Four H2 files. A service may store another service's UUID. Nobody runs SQL against another module's tables.

Q: Show me Git and CI.

A: One repo. Feature branches and pull requests. GitHub Actions runs ./mvnw -B verify on push/PR.

Q: Who wrote this service? Prove it.

A: Open YOUR controller and one test. Name your folder under services/. Commits must be your GitHub user.

If they try to trap you

Q: Where is the mobile app?

A: Out of scope. Marks are on APIs, Swagger, and Postman.

Q: Can I book a real taxi?

A: No. Fictional data, simulated location and payment.

Q: Did AI write this?

A: We used tools under institute policy and can explain every line. Declare AI in the report appendix.

9. One-page cheat (memorise)

```
Java 17 + Spring Boot. Four services, four DBs.
JWT from Account. Roles: PASSENGER, DRIVER, ADMIN.
Fare: 150 + km*80 + min*5. Distance = hash of place names.
Assign: REST eligible, first/dummy-nearest. Colombo seeded driver.
States: REQUESTED > ASSIGNED > ACCEPTED > IN_PROGRESS > COMPLETED (+ CANCELLED).
401 no token. 409 no driver. 400 bad transition. 0000 pay fails.
Users: passenger1 / driver1 / admin1      password=password
Chrome http://127.0.0.1:8081 8082 8083 8084 /swagger-ui.html
```

Print this PDF. Each member bookmarks their section. Practise the happy path once with a timer. If Swagger says connection refused, the Java process is not running — start your service, wait for Started ...Application, then refresh Chrome.

